NORTHWESTERN UNIVERSITY

Contract Checking as Effects

A DISSERTATION

SUBMITTED TO THE GRADUATE SCHOOL

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

Master of Science

Field of Computer Science

By

Anh Khoa Nguyen

EVANSTON, ILLINOIS

December 2026

ABSTRACT

Contract Checking as Effects

Anh Khoa Nguyen

nice

Acknowledgements

Text for acknowledgments goes here. You can thank your advisors, peers, and family.

Preface

This is the preface (optional).

Dedication

This is the dedication (optional).

Table of Contents

List of Tables

List of Figures

CHAPTER 1

Introduction

Software contracts, or *behavioral contracts*, allow programmers to attach assertions to values, and functions. These assertions are runtime guarantees, on violation, an error is thrown, blaming the violating party. Contracts are most useful when placed on a function, enforcing the function's input and output values. While first-order functional contracts are useful, they are too simple. Higher-order functional contracts allows programmers to enforce contracts for higher-order functions. Dependent contracts are functional contracts allowing usage of the function argument during post-condition contract checking.

Recently, algebraic effect handlers has gained a large attraction, and is adopted in OCaml since version 5.0. Effect handlers open a new way to identify effects and handling them. Handlers are defined to handle one or many effects. A handler for an effect can stop the program or continue with some result. Handlers do not have state by default, but it can be implemented by applying the new states to the continuation closure, and let the underlying computation receives a state argument.

Combining the contract system and effect handlers is a new approach. Effectful Contract [9] demonstrates how both systems can be combined, providing contracts for effectful code. Contracts in their system remain a language feature. In this dissertation, instead of combining the effects and contracts, we ask ourselves whether contract checking itself can become an effect, and reuse the capabilities of handlers.

> *Contracts checking can be encoded as side-effects and benefited from context sharing when handled using effect handlers.*

1.1 Contributions

This thesis explores the contracts checking as effects'' approach to implementing software contracts. Through experiments, we show that this approach accomodates functional contracts, dependent contracts, and the challenging higher-order contracts. We also discovered that as effects, their contexts can be shared if they are under the same handler. Sharing contracts context opens new capabilities to optimizations and contract state sharing. This disseration shows the above idea by providing a compiler model between CPCF [2] and System F^{ε} [14]. We also provide an implementation in OCaml as a practical implementation.

The rest of the thesis is organized as follows: Chapter ref{chap:contracts} and Chapter ref{chap:effects} introduces the two formal languages Contracts based on CPCF, and `Effects` based on System F^{ε}; Chapter ref{chap:compiler} discusses a theoretical compiler from Contracts to `Effects` as a demonstration to contract checking as effects''; Chapter ref{chap:ocaml} introduces the OCaml language and its native support for effect handlers; Chapter ref{chap:implementation} provides the implementation of the theoretical compiler in OCaml using PPX rewritter; Chapter ref{chap:compare} compares our implementation in OCaml to the Racket's contract system; Chapter ref{chap:final} concludes our remarks.

CHAPTER 2

Formal Language Models

2.1 Core Language

We define **Core** language and its syntax in Figure 1. The language extends the PCF language with tuples and mutable cells. This will be the base language for both Contracts and Effects.

Core

$$
\begin{array}{rcl}
\tau & ::= & o \\
& | & \tau \to \tau \\
& | & \langle \tau, \tau \rangle \\
& | & \mathsf{ref}(\tau) \\
\\
e & ::= & 0 \mid -1 \mid 1 \mid ... \mid \mathrm{tt} \mid \mathrm{ff} \mid \mathrm{x} \\
& | & \lambda x : \tau . e \mid \mu x : \tau . e \\
& | & e + e \mid e - e \mid e \wedge e \mid e \vee e \\
& | & e\ e \mid \text{if } e \text{ then } e \text{ else } e \\
& | & \mathsf{zero?}(e) \\
& | & \mathsf{inj.l}(e) \mid \mathsf{inj.r}(e) \\
& | & \mathsf{new}(e) \mid \mathsf{get}(e) \mid \mathsf{set}(e, e)
\end{array}
\qquad
\begin{array}{rcl}
o & ::= & \mathrm{num} \\
& | & \mathrm{bool} \\
\\
v & ::= & 0 \mid -1 \mid 1 \mid ... \mid \mathrm{tt} \mid \mathrm{ff} \mid \mathrm{x} \\
& | & \lambda x . e \mid \mathsf{loc} \\
\\
E & ::= & \square \\
& | & E + e \mid v + E \mid E - e \mid v - E \\
& | & E \wedge e \mid v \wedge E \mid E \vee e \mid v \vee E \\
& | & E\ e \mid v\ E \mid \text{if } E \text{ then } e \text{ else } e \mid \mathsf{zero?}(E) \\
& | & \mathsf{inj.l}(E) \mid \mathsf{inj.r}(E) \\
& | & \mathsf{new}(E) \mid \mathsf{get}(E) \mid \mathsf{set}(E, e) \mid \mathsf{set}(v, E)
\end{array}
$$

Figure 1: Core Language

2.2 Contracts Language

2.2.1 Example

2.2.2 Formal

We define Contracts, in Figure 2, by extending the Core language with contracts. Contracts protect an expression, ensuring that the evaluated value conform to the contracts defined. Contracts are defined for all possible types of value. Base values, including booleans and

numbers, are trivial to enfore a contract on them. While functions need to check for both its argument and return result. Not only that, functions are first-class citizen in the language, and we can form higher-order functions. A contract for function may want to "see" the argument when checking the result, such is the idea of dependent contracts. Both contracts higher-order functions, and dependent contracts semantics are discussed in [4].

The language supports contracts for tuples. Their contracts will be applied individually during execution. Mutable cells contracts follow the work of [2], where a special contract type $\mathsf{ref/c}(\kappa)$ is used to protect the cell's value.

$\boxed{\mathsf{Contracts}}$ extends **Core**

$$
\begin{array}{lcl@{\qquad}lcl}
\tau & ::= & \ldots & \kappa & ::= & \mathsf{flat}(e) \mid \kappa \to \kappa \mid \kappa \overset{d}{\mapsto} (\lambda x : \tau.\kappa) \\
 & \mid & \mathsf{con}(\tau) & & \mid & \langle \kappa, \kappa \rangle \mid \mathsf{ref/c}(\kappa) \\
\\
e & ::= & \ldots & v & ::= & \ldots \\
 & \mid & \mathsf{mon}_j^{k,l}(\kappa, e) & & \mid & \mathsf{guard}_j^{k,l}(\kappa, v) \\
 & \mid & \mathsf{blame}_p^l & & & \\
 & & & E & ::= & \ldots \\
 & & & & \mid & \mathsf{mon}_j^{k,l}(\kappa, E)
\end{array}
$$

Figure 2: Contracts language

When a contract violation occurs, a blame is thrown with the party, caller or callee. The semantics for correct blaming has been studied under [2] and [3]. Therefore, we use *indy* semantics for dependent contracts.

The type system is straightforward, readers interested in the type rules for Contracts can consult the Appendix. We provide the semantics for the language in Figure 3.

$$\mathsf{guard}_j^{k,l}(\mathrm{tt}, v) \longrightarrow v \quad \text{Guard-True}$$

$$\mathsf{guard}_j^{k,l}(\mathrm{ff}, v) \longrightarrow \mathsf{blame}_j^k \quad \text{Guard-False}$$

$$\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \longrightarrow \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \quad \text{Guard-Func}$$

$$\mathsf{guard}_j^{k,l}\left(\kappa_1 \xrightarrow{d} (\lambda x : \tau.\kappa_2), v\right) \longrightarrow \begin{array}{l}\text{let } x_1 = \mathsf{mon}_j^{l,k}(\kappa_1, x) \\ \text{let } x_2 = \mathsf{mon}_j^{l,j}(\kappa_1, x) \\ \lambda x.\mathsf{mon}_j^{k,l}(\{x_2/x\}\kappa_2, v\ x_1)\end{array} \quad \text{Guard-Dep}$$

$$\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v) \longrightarrow \mathsf{guard}_j^{k,l}(e\ v, v) \quad \text{Mon-Flat}$$

$$\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v) \longrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \quad \text{Mon-Func}$$

$$\mathsf{mon}_j^{k,l}\left(\kappa_1 \xrightarrow{d} (\lambda x : \tau.\kappa_2), v\right) \longrightarrow \mathsf{guard}_j^{k,l}\left(\kappa_1 \xrightarrow{d} (\lambda x : \tau.\kappa_2), v\right) \quad \text{Mon-Dep}$$

$$\mathsf{mon}_j^{k,l}(\langle \kappa_1, \kappa_2\rangle, \langle v_1, v_2\rangle) \longrightarrow \langle \mathsf{mon}_j^{k,l}(\kappa_1, v_1), \mathsf{mon}_j^{k,l}(\kappa_2, v_2)\rangle \quad \text{Mon-Tuple}$$

$$\mathsf{mon}_j^{k,l}(\mathsf{ref/c}(\kappa), v) \longrightarrow \mathsf{guard}_j^{k,l}(\kappa, v) \quad \text{Mon-Cell}$$

$$\mathsf{get}\left(\mathsf{guard}_j^{k,l}(\kappa, v)\right) \longrightarrow \mathsf{mon}_j^{k,l}(\kappa, \mathsf{get}(v)) \quad \text{Mon-Get}$$

$$\mathsf{set}\left(\mathsf{guard}_j^{k,l}(\kappa, v_1), v_2\right) \longrightarrow \lambda x.\mathsf{mon}_j^{k,l}\left(\mathsf{ref/c}(\kappa), \mathsf{set}\left(v_1, \mathsf{mon}_j^{l,k}(\kappa, x)\right)\right)\ v_2 \quad \text{Mon-Set}$$

$$\frac{E \neq \square}{E\left[\mathsf{blame}_p^l\right] \longrightarrow \mathsf{blame}_p^l}\ \text{Blame}$$

Figure 3: Reduction Rules for Contracts

2.3 Effects Language

2.3.1 Example

2.3.2 Formal

We continue to extend the **Core** language with effects and effect handlers in Listing 1. This is done by typing the effects separately from the base types. This practice has been introduced to formalize the Koka[1] language. The approach we take here resembles the

[1] https://koka-lang.github.io/

System F^{ε} language introduced in [14]. We added polymorphism through type application, and kinds to **Core** language. Two special kinds are added to represent effects (eff) and effect labels (lab). A label $l \in \texttt{lab}$ is a group of operations. And an effect $e \in \texttt{eff}$ is composed by chaining lab together with other effects. An empty effect $\langle\rangle$ defines no labels, therefore no effects can be performed. In otherwords, eff defines the set of effects that can be performed. A function can now produce some effects $\varepsilon \in \texttt{eff}$, and is represented in its type $\tau_1 \to^{\varepsilon} \tau_2$.

The language is then extended with handlers, handler installation ($\texttt{handle}\ h\ e$), and calling/performing an effect ($\texttt{perform}\ \texttt{op}\ \tau$). Handlers are a set of operations distinguishable by the operation's name. A handler denotes a single effect usually by its label $l \in \texttt{lab}$. Performing an effect is analogous function application, therefore $\texttt{perform}\ \texttt{op}\ \tau$ is treated as a value, where application for it has a distinct reduction rule. The evaluation context is thus extended to support handler installation.

$\boxed{\texttt{Effects}}$ extends **Core**

$$
\begin{array}{rcl}
\tau & ::= & \ldots \\
& | & \alpha^k \mid c^k \tau \ldots \mid \tau \to^{\varepsilon} \tau \mid \forall \alpha^k . \tau \\
\\
k & ::= & * \\
& | & k \to k \\
& | & \texttt{eff} \\
& | & \texttt{lab} \\
\\
e & ::= & \ldots \\
& | & e[\tau] \\
& | & \texttt{handle}\ h\ e \\
& | & \texttt{blame}_p \\
\\
h & ::= & \{\texttt{op}_1 \to f_1, \ldots, \texttt{op}_n \to f_n\}
\end{array}
\qquad
\begin{array}{rcl}
v & ::= & \ldots \\
& | & \Lambda \alpha^k . v \\
& | & \texttt{handler}\ h \\
& | & \texttt{perform}\ \texttt{op}\ \tau \\
\\
E & ::= & \ldots \\
& | & E[\tau] \\
& | & \texttt{handle}\ h\ E \\
\\
F & ::= & \square \\
& | & F + e \mid v + F \mid F \wedge e \mid v \wedge F \mid F \vee e \mid v \vee \\
& & F \\
& | & F\ e \mid v\ F \\
& | & \text{if } F \text{ then } e \text{ else } e \\
& | & F[\tau]
\end{array}
$$

Listing 1: Effects language

Typing for an effect language requires some consideration. Following the works in [14], we define the type judgement relationship $\Delta \vdash e : \tau \mid \varepsilon$ between a type context Δ, an expression e and assert it against type τ with some possible effects ε. Variables and base values do not have effects, they can take any effects. For this reason, we define $\Delta \vdash_{\text{val}} v : \tau$. Handlers are typed using $\Delta \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon$, that it handles a *single* effect l, other effects in ε remains unhandled, and all operations returns τ. All type rules unique to `Effects` is presented in Figure 4.

$$\frac{\cdots}{\cdots}\,\text{SomeRule}$$

Figure 4: Type Rules for `Effects`

Lastly the semantic model of `Effects` is presented in Listing 2. A handler can be installed to a function as a shorthand, where it applies the function with a unit argument, which to be ignored. The reduction for handling an effect calling is encoded as *deep handler*. Supporting the shallow should be straightforward by not installing the handler again in the continuation k. There is little advantage to support shallow handlers, therefore, only deep handlers are used. While looking for the handler, we make sure we get the innermost handler that can handle the operation.

$$(\Lambda\alpha^k.v)[\tau] \longrightarrow v[\alpha := \tau] \quad \text{Step-E-TApp}$$

$$\texttt{handler}\ h\ v \longrightarrow \texttt{handle}\ h\ (v\ ()) \quad \text{Step-E-Handler}$$

$$\texttt{handle}\ h\ v \longrightarrow v \quad \text{Step-E-Return}$$

$$\frac{\texttt{op} \notin \texttt{bop}(E) \wedge (\texttt{op} \to f) \in h \qquad \texttt{op} : \forall\alpha.\tau_1 \to \tau_2 \in \Sigma(l) \qquad k = \lambda x : \tau_2[\alpha := \tau].\ \texttt{handle}\ h\ E[x]}{\texttt{handle}\ h\ E[\texttt{perform}\ \texttt{op}\ \tau\ v] \longrightarrow f[\tau]\ v\ k} \text{Step-E-Perform}$$

$$\frac{E \neq \square}{E\left[\texttt{blame}_p\right] \longrightarrow \texttt{blame}_p} \text{Effect-Error}$$

Helper functions

$$\begin{aligned}
\texttt{bop}(\square) &= \varnothing \\
\texttt{bop}(E\ e) &= \texttt{bop}(E) \\
&\vdots \\
\texttt{bop}(\texttt{handle}\ h\ E) &= \texttt{bop}(E) \cup \{\texttt{op} \mid (\texttt{op} \to f) \in h\}
\end{aligned}$$

Listing 2: Reduction Rules for `Effects`

Bibliography

[1] Andrej Bauer and Matija Pretnar. 2014. An Effect System for Algebraic Effects and Handlers. *Log. Methods Comput. Sci.* 10, 4 (2014). https://doi.org/10.2168/LMCS-10(4:9)2014

[2] Christos Dimoulas and Matthias Felleisen. 2011. On contract satisfaction in a higher-order world. *ACM Trans. Program. Lang. Syst.* 33, 5 (2011), 16:1–16:29. https://doi.org/10.1145/2039346.2039348

[3] Christos Dimoulas, Sam Tobin-Hochstadt, and Matthias Felleisen. 2012. Complete Monitors for Behavioral Contracts. In *Programming Languages and Systems - 21st European Symposium on Programming, ESOP 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings (Lecture Notes in Computer Science)*, 2012. Springer, 214–233. https://doi.org/10.1007/978-3-642-28869-2_11

[4] Robert Bruce Findler and Matthias Felleisen. 2013. ICFP 2002: Contracts for higher-order functions. *ACM SIGPLAN Notices* 48, 4S (2013), 34–45. https://doi.org/10.1145/2502508.2502521

[5] Oleg Kiselyov and KC Sivaramakrishnan. 2018. Eff directly in OCaml. *arXiv preprint arXiv:1812.11664* (2018).

[6] Daan Leijen. 2016. *Algebraic Effects for Functional Programming.* Retrieved from https://www.microsoft.com/en-us/research/publication/algebraic-effects-for-functional-programming/

[7] Paul Blain Levy. 2013. Call-by-push-value. Doctoral dissertation.

[8] Bertrand Meyer. 1992. The eiffel programming language. *See http://www. eiffel. com* (1992).

[9] Cameron Moy, Christos Dimoulas, and Matthias Felleisen. 2024. Effectful Software Contracts. *Proc. ACM Program. Lang.* 8, POPL (2024), 2639–2666. https://doi.org/10.1145/3632930

[10] Gordon D. Plotkin and John Power. 2003. Algebraic Operations and Generic Effects. *Appl. Categorical Struct.* 11, 1 (2003), 69–94. https://doi.org/10.1023/A:1023064908962

[11] Matija Pretnar. 2010. Logic and handling of algebraic effects. Doctoral dissertation. Retrieved from https://hdl.handle.net/1842/4611

[12] Matija Pretnar. 2015. An Introduction to Algebraic Effects and Handlers. Invited tutorial paper. In *The 31st Conference on the Mathematical Foundations of Programming Semantics, MFPS 2015, Nijmegen, The Netherlands, June 22-25, 2015 (Electronic Notes in Theoretical Computer Science)*, 2015. Elsevier, 19–35. https://doi.org/10.1016/J.ENTCS.2015.12.003

[13] K. C. Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, and Anil Madhavapeddy. 2021. Retrofitting Effect Handlers onto OCaml. *CoRR* (2021). Retrieved from https://arxiv.org/abs/2104.00250

[14] Ningning Xie, Jonathan Immanuel Brachthäuser, Daniel Hillerström, Philipp Schuster, and Daan Leijen. 2020. Effect handlers, evidently. *Proc. ACM Program. Lang.* 4, ICFP (2020), 99:1–99:29. https://doi.org/10.1145/3408981

[15] Dana N Xu. 2014. Dynamic Contract Checking for OCaml.

CHAPTER A

Contract Erasure

CHAPTER B

Effects Safety